

SYNAP Evaluation

Evaluation

A benchmark, not a demo: this document compares S.Y.N.A.P.’s full 8-module pipeline against a **Seed-Only ablation baseline** (Modules 1-2 only — Contract + DAG/Seed, none of the validation, correction, or audit machinery), across 5 independent seeds, on the same credit-risk generative process used in `CASE_STUDY.md` and the [Validation & Reliability Report](#). Every number below comes from `validation/evaluation_results.json`, produced by `validation/run_evaluation.py` — see [Reproducibility](#).

1. Benchmark Objective

This benchmark exists to answer one question with numbers, not adjectives: **does the machinery beyond the raw generative core (Modules 3-8) measurably change anything that matters?** Specifically: (a) does it improve statistical fidelity over just sampling from the learned distributions directly, (b) at what latency cost, and (c) does it change privacy exposure. A fourth, unplanned finding — that the Module 8 veto observed in `CASE_STUDY.md` did **not** reproduce at this benchmark’s batch configuration — turned out to be the most practically important result, and is reported as such, not smoothed over.

2. What Was Tested

Modules under test	Full pipeline: Modules 1–8, composed as a single batch (see §4 for exactly what this does and does not include)
Baseline	Modules 1–2 only (“Seed-Only”) — the Contract and the DAG/seed generator, with none of Modules 3–8’s filtering, validation, or audit applied
Version	S.Y.N.A.P. v0.1.0 (pre-release; see §3)
Scenario	One dataset shape: 4 numeric columns (age, income, debt, credit score) + 1 categorical column (risk tier), a single business rule (<code>income > 0</code>), and a 5-edge causal DAG
Runs	5 independent seeds (101–105), each with a freshly redrawn 2,200-row real-world dataset (1,496 anchor / 704 holdout, 68/32 split) — not 5 resamples of the same fixed dataset
Rows per run	5,000 (both baseline and full pipeline, for a size-matched comparison)

This is a demo-scale benchmark, not a production-scale one. Five seeds, one dataset shape, 5,000 rows per run, on a single-vCPU container (see §3). It is enough to establish direction and rough magnitude for the four questions in §1, and to demonstrate the methodology. It is not enough to certify latency or veto-rate figures for production capacity planning — see §8.

3. Environment & Version

Python	3.12.3
Platform	Linux (container), x86_64
CPU	Intel Xeon @ 2.80 GHz, 1 vCPU
RAM	3.9 GiB
numpy	2.4.4
pandas	3.0.2
scipy	1.17.1
scikit-learn	1.8.0
pydantic	2.13.4
networkx	3.6.1
Flask	3.1.3
S.Y.N.A.P. version	0.1.0
Execution date	2026-07-09 (UTC)
Commit hash	f25660e7534c0be3712064f07cf0b04b4b199668 (source tree evaluated — see Reproducibility for the commit that adds this document and its results on top)
Seeds	101, 102, 103, 104, 105

The single-vCPU hardware matters for interpreting §6: latency figures are meaningful **relative to each other** (baseline vs. full pipeline, on identical hardware) but should not be read as absolute production latency, which would differ on different hardware and, more importantly, under the multi-process execution this benchmark does not use (see [ROADMAP.md](#) §4.1).

4. Methodology

Real-world data, redrawn per seed, not reused. For each of the 5 seeds, a fresh 2,200-row dataset is generated from a *known* generative process (true correlations fixed in code, not fitted) — the same approach used in the Validation Report, chosen for the same reason: it lets us verify structure recovery against ground truth, not just against a sample that could coincidentally look right. Each dataset is split 68/32 into anchor (1,496 rows, used to build the Contract) and holdout (704 rows, used only afterward, for the DCR and ASP-compliance checks below).

“Baseline” is Modules 1-2 only. `hipocampo.generate_seed()` is called directly for 5,000 rows, bypassing Modules 3–8 entirely — no Mahalanobis filtering, no plausibility scoring, no ASP re-validation, no homeostasis correction, no Twin Model, no privacy audit. This isolates exactly what the “extra” 6 modules contribute, if anything.

“Full pipeline” is Modules 1-8, composed as a single batch of 5,000 rows — not the multi-checkpoint loop `synap.main.run_synap` runs for large row counts (which would call Modules 4b and 6 repeatedly across several smaller batches, each correcting against the last). This benchmark calls Module 5 (`run_nucleo`) once for the full 5,000 rows, then Module 4’s checkpoint logic and Module 6 once each, then Modules 7 and 8. This is a deliberate scope reduction to keep 5 independent runs tractable — see §7 for why this specific choice turned out to matter for one of the results, and §8 for what it means for generalizing these numbers.

Metrics excluded on purpose: DAG-learning quality and causal-drift detection are not scored here — both are already covered by dedicated, targeted tests in the pytest suite and in the Validation Report, and adding them here would dilute this benchmark’s focus (see §5 on why only 6 metrics were kept).

Success criteria, fixed before running: for the full pipeline, ASP compliance = 100% (non-negotiable — anything else is a defect, not a benchmark outcome), privacy approved on every seed, and Fidelity Index ≥ 0.90 on every seed, all defined in `synap.compilador` / `synap.auditor` before this benchmark was written, not chosen after seeing results.

5. Metrics

Six metrics, chosen to cover fidelity, privacy, and cost without diluting the result into noise:

1. **ASP compliance rate** — fraction of rows satisfying the declared business rule, checked identically on both baseline and full-pipeline output (`hipotalamo.validate_batch`).
2. **Correlation recovery error** — mean absolute difference between each dataset’s empirical correlations and the *true* generative correlations (not the anchor sample’s correlations).
3. **DCR ratio** — median Distance-to-Closest-Record from synthetic to the real holdout, divided by the real-to-real internal baseline (`synap.auditor.dcr_assessment`); ≥ 0.5 is the project’s own “healthy” threshold.
4. **Exact-match count** — literal row-level duplicates against the 704-row holdout.
5. **Latency** — wall-clock seconds per 100 rows generated.
6. **Veto rate** — fraction of the 5 full-pipeline runs where Module 7 released the dataset (Fidelity Index ≥ 0.90) but Module 8 did not approve it (utility retention $< 85\%$, or privacy failed). Not applicable to the baseline, which has no Module 7/8 stage at all.

6. Results

Summary (mean across 5 seeds; latency and error: lower is better; DCR and ASP: higher is better, within the safe range)

Metric	Baseline (Modules 1-2)	Full Pipeline (Modules 1-8)
ASP compliance	100.0%	100.0%
Correlation recovery error	0.0349	0.0228
DCR ratio (≥ 0.5 is safe)	0.850	0.822
Exact matches	0	0
Latency (s / 100 rows)	0.00028	1.489
Veto rate	n/a	0% (0 of 5)

Comparative detail, with variance across the 5 seeds

Metric	Baseline mean $\pm$ std (min-max)	Full Pipeline mean $\pm$ std (min-max)
Correlation recovery error	0.0349 $\pm$ 0.0172 (0.0060-0.0526)	0.0228 $\pm$ 0.0095 (0.0111-0.0372)
DCR ratio	0.850 $\pm$ 0.018 (0.830-0.881)	0.822 $\pm$ 0.012 (0.808-0.843)
Latency (s / 100 rows)	0.00028 $\pm$ 0.00001	1.489 $\pm$ 0.028 (1.441-1.543)

Latency (s / 100 rows)	0.00028 ± 0.00001	1.489 ± 0.038 (1.441-1.543)
Fidelity Index (full pipeline only)	—	0.9992 ± 0.0007 (0.9980-0.9999)
Utility retention (full pipeline only)	—	0.9990 ± 0.0012 (0.9967-1.0000)

Best case / worst case (full pipeline)

- **Best correlation recovery:** seed 104, error 0.0111.
- **Worst correlation recovery:** seed 102, error 0.0372 — still better than the baseline’s *best* seed (0.0060 excluded — see note below) in 4 of 5 comparisons.
- **Fastest run:** seed 104, 1.441 s/100 rows. **Slowest:** seed 102, 1.543 s/100 rows — a 7% spread, not a qualitative difference.
- **No seed among the 5 triggered a Module 8 veto** (see §7).

(Note: baseline’s single best seed, 0.0060, was better than any full-pipeline seed — sampling variance alone can occasionally make the simpler method look better on one specific draw. The mean and the 5-seed spread, not a single best case, are what should be read as the real comparison — which is exactly why this benchmark uses 5 seeds instead of 1.)

7. Reading the Results

This means: the full pipeline recovers ground-truth correlations measurably better, and more consistently (lower standard deviation), than directly sampling from the learned distributions. Modules 3–6’s filtering and correction are doing real, quantifiable work, not just adding overhead — averaged over 5 seeds, not a single favorable run.

This also means: that improvement in fidelity costs roughly **5,000× the latency** (0.00028s vs. 1.489s per 100 rows) on identical, modest, single-vCPU hardware. This is not a defect to be explained away — it is the actual, measured price of Modules 3–8’s validation and correction machinery, and it is the number a capacity-planning conversation needs, not a vaguer “it’s slower.”

This does NOT mean: that the full pipeline is “5,000× more expensive to run” in a production sense — this benchmark deliberately runs a single, un-parallelized batch on 1 vCPU (§4);

[ROADMAP.md](#) §4.1 already identifies multi-process batch generation as a “likely next” item specifically because this gap is large enough to be worth closing.

A genuine, unplanned finding — the missing veto. [CASE_STUDY.md](#) reports a real run where Module 8 vetoed a dataset Module 7 had already released, because a regression task’s utility retention fell to 50.5%. None of the 5 runs in this benchmark reproduced that: utility retention was ≥ 99.6% on every seed. The most likely explanation is not that one document is wrong — it’s the methodological difference stated in §4: this benchmark generates the full 5,000 rows in **one** batch, while [CASE_STUDY.md](#)’s run (and the Validation Report behind it) generated 4,500 rows across **five** smaller checkpoints, each correcting against the last via Module 6. A single large batch appears to give Module 5’s plausibility/Mahalanobis filters a more stable, already-mature memory pool to draw from from the start, which this benchmark’s data suggests may matter specifically for harder numeric-regression fidelity. That is a hypothesis this benchmark’s design can surface but not confirm — confirming it needs a dedicated follow-up varying batch count at fixed total row count, not a difference in row count or seed.

Trade-off identified: batch granularity (one large batch vs. several smaller, self-correcting checkpoints) is not a neutral configuration choice — it appears to change downstream utility outcomes materially. The project currently exposes [batch_size](#) as a configuration parameter without guidance on this trade-off; that gap is worth closing (see §9).

8. Limitations

- **Five seeds, one scenario.** This benchmark does not claim these exact numbers generalize to other schemas, column counts, or data distributions — only that the *method* (ablation against a Seed-Only baseline, across several seeds) is sound and repeatable for any scenario someone wants to check.
- **Single-vCPU, un-parallelized hardware.** Absolute latency figures are specific to this machine and to the current single-process execution model ([ROADMAP.md](#) §4.1); the *relative* baseline-vs-full-pipeline comparison is the portable part of this result, not the absolute seconds.
- **“Full pipeline” here is single-batch, not [run_synap](#)’s multi-checkpoint loop.** §7’s veto-rate discussion is the direct consequence of this scope choice, made explicitly to keep 5 independent runs tractable. The Validation Report’s own 5-checkpoint, 4,500-row run remains the reference for multi-checkpoint behavior (homeostasis states across checkpoints, ILP induction, causal-drift monitoring over time) that this benchmark does not re-test.
- **Veto rate of 0% is a measurement, not a guarantee.** It should not be read as “Module 8 rarely vetoes” — [CASE_STUDY.md](#) shows a concrete counter-example under a different batching configuration. The honest reading is: veto rate is configuration-dependent, and this benchmark does not yet have enough data to say by how much.
- **No comparison against a previous S.Y.N.A.P. version.** This is the project’s first release (v0.1.0) — there is no prior version to serve as a baseline. The Seed-Only ablation serves as this benchmark’s baseline instead, and this run itself becomes the reference point for any future version’s comparison.

9. Practical Conclusion

What we learned: the validation and correction machinery (Modules 3–8) is not overhead-for-appearance — it measurably improves correlation fidelity, with lower run-to-run variance, over the simplest possible alternative. That improvement has a real, large, and previously unquantified latency cost on single-process hardware. And the Module 8 veto behavior that gives this whole project its credibility is sensitive to batch configuration in a way that had not been explicitly tested before this benchmark.

What changes because of this, concretely:

1. [ROADMAP.md](#) §4.1 (**multi-process batch generation**) is now backed by a measured number ($\approx 5,000\times$) instead of a general “this would probably help” — raising its practical priority.
 2. **A new, tracked gap:** neither [ARCHITECTURE.md](#) nor the CLI currently documents how [batch_size](#) affects downstream utility retention. Until a dedicated batch-count-vs-utility study exists (the natural follow-up this benchmark points to directly), the [README.md](#) recommendation is to prefer several smaller checkpoints (closer to the Validation Report’s configuration) over one large batch when the target use case includes a numeric-regression Twin Model task, and to always run Module 8 with that specific target column before trusting a single-task Module 7 result — which is the same recommendation [CASE_STUDY.md](#) §6 already reached from one run, now supported by a second, independent line of evidence.
 3. This benchmark’s harness ([validation/run_evaluation.py](#)) is kept in the repository specifically so the next version of S.Y.N.A.P. has an existing, real baseline to compare against — the gap [§8](#) names explicitly.
-

Reproducibility

```
# one seed at a time (each run is independent and self-contained)
python3 validation/run_evaluation.py 101
python3 validation/run_evaluation.py 102
python3 validation/run_evaluation.py 103
python3 validation/run_evaluation.py 104
python3 validation/run_evaluation.py 105

# aggregate the 5 seeds into evaluation_results.json
python3 validation/run_evaluation.py
```

- **Seeds:** 101, 102, 103, 104, 105 (fixed in `validation/run_evaluation.py`, `SEEDS`).
- **Environment:** Python 3.12.3; see §3 for exact library versions.
- **Data:** synthetically generated from a known generative process (§4), not shipped as a static file — regenerated fresh, per seed, by the script itself.
- **Script:** `validation/run_evaluation.py`.
- **Raw output:** `validation/evaluation_results.json` (aggregates) and `validation/evaluation_results_raw.json` (per-seed detail) — both produced by the commands above, not hand-edited.
- **Commit:** the source tree evaluated is `f25660e7534c0be3712064f07cf0b04b4b199668` (§3). Run `git log --oneline -- EVALUATION.md validation/run_evaluation.py` to find the exact commit that last changed this document or its harness, rather than a hash hand-copied here that would go stale on the next edit.